

Last-Come-First-Served Hashing*

PATRICIO V. POBLETE

*Departamento de Ciencias de la Computacion, Universidad de Chile,
Casilla 2777, Santiago, Chile*

AND

J. IAN MUNRO

*Data Structuring Group, Department of Computer Science, University of Waterloo,
Waterloo, Ontario, Canada N2L 3G1*

Received November 10, 1987; accepted March 29, 1988

We introduce a new heuristic for resolving collisions in open addressing hash tables, *last-come-first-served hashing*. The analysis shows that its expected search time is the same as that of the standard method, but its variance is much smaller. This makes the search algorithm much more reliable, and it may also be used to conduct a mode-centered search, further decreasing the search time. © 1989 Academic Press, Inc.

1. INTRODUCTION

When keys are inserted into a hash table, collisions are likely to occur, even when the number of elements, n , is small compared to the table size, m . Two popular ways of handling collisions are *chaining* and *open addressing* [15]. In this paper, we are interested in the latter class of methods.

Under open addressing hashing, a key K is mapped to a sequence of table addresses $\langle h_i(K) \rangle_{i \geq 1}$, such that $\langle h_1(K), \dots, h_m(K) \rangle$ is a permutation of the numbers $\{0, \dots, m-1\}$. Under the standard insertion algorithm, to add a new key K to the table we probe locations $h_1(K)$, $h_2(K)$, etc. until we find an empty one, and then store K in that location.

In practice, the probe sequence is usually generated using *double hashing*: each key is mapped to two values, $r(K) \in \{0, \dots, m-1\}$ and $s(K) \in$

*This work was supported under National Science and Engineering Research Council of Canada Grant A-8237 while the first author was on leave at the University of Waterloo.

$\{1, \dots, m-1\}$, and the probe sequence is constructed as

$$h_i(K) = (r(K) + (i-1)s(K)) \bmod m.$$

The simplest way to guarantee that $\langle h_1(K), \dots, h_m(K) \rangle$ is a permutation is to let m be a prime.

For analytical purposes, two idealized methods are usually considered instead of double hashing. Under the *uniform probing* model, we assume $\langle h_1(K), \dots, h_m(K) \rangle$ is a random permutation of $\{0, \dots, m-1\}$ (this corresponds to a model of sampling without replacement). The *random probing* model asserts that each $h_i(K)$ in the (potentially infinite) probe sequence may take any of the values in $\{0, \dots, m-1\}$ with equal probability (this corresponds to sampling with replacement). Both models have been found empirically to approximate closely the performance of double hashing, for large m , even when n is quite close to m , and it has been proven [13] that when $n < 0.319 \dots m$, double hashing is asymptotically equivalent to uniform probing.

Our main performance measures for hashing algorithms will be represented by the following random variables:

- A_n , the number of probes needed to perform a successful search for a random element (assuming all n elements are equally likely), and
- L_n , the maximum number of probes needed to find an element present in the table.

Other performance measures of interest are the unsuccessful search time, A'_n , and the time to perform an insertion, I_n .

The results below summarize the behavior of the uniform probing open addressing hashing method [10, 15]. We give the exact formula for a table of size m with n keys and the asymptotic expressions for the case of a full table ($n = m$) and the case of an α -full table (i.e., a table with $n = \alpha m$ elements, for some fixed parameter α , $0 \leq \alpha < 1$; α is called the “load factor” of the table).

Under the uniform probing model, the expected successful search time is

$$\begin{aligned} \mathbf{E}[A_n] &= \frac{m+1}{n} (H_{m+1} - H_{m-n+1}) \\ \mathbf{E}[A_m] &= \frac{m+1}{m} (H_{m+1} - 1) = \ln m + \gamma - 1 + O\left(\frac{1}{m}\right) \\ \mathbf{E}[A_{\alpha m}] &= \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \quad 0 \leq \alpha < 1, \end{aligned}$$

and the variance of the successful search time is given by

$$\begin{aligned} V[A_n] &= 2 \frac{m+1}{m+2-n} - \frac{(m+1)^2}{n^2} (H_{m+1} - H_{m+1-n})^2 \\ &\quad - \frac{m+1}{n} (H_{m+1} - H_{m+1-n}) \\ V[A_m] &= m+1 - \frac{m+1}{m} (H_{m+1} - 1) - \frac{(m+1)^2}{m^2} (H_{m+1} - 1)^2 \\ &= m - \ln^2 m + (1 - 2\gamma) \ln m + (1 + \gamma - \gamma^2) + O\left(\frac{1}{m}\right) \\ V[A_{\alpha m}] &= \frac{2}{1-\alpha} - \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) - \frac{1}{\alpha^2} \ln^2\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \\ &\quad 0 \leq \alpha < 1. \end{aligned}$$

Lyon [16] made the observation that a search can be stopped, and considered unsuccessful, if its length exceeds L_n for that table. Of course, this is unlikely to be helpful if L_n is large so we are interested in studying the behavior of this behavior of this random variable. Gonnet [9] has proved that

$$E[L_{\alpha m}] = -\log_{\alpha} m - \log_{\alpha}(-\log_{\alpha} m) + O(1)$$

and, for a full table,

$$E[L_m] = 0.631587464 \cdots \times m + O(1).$$

Consider now the same algorithm under the assumption of random probing. Intuitively, the effect of sampling the address space with replacement should be an increase in the retrieval time and in its variance, due to probing some locations more than once. However, the following results show that the difference is very small for large m :

$$\begin{aligned} E[A_n] &= \frac{m}{n} (H_m - H_{m-n}) \\ E[A_m] &= H_m = \ln m + \gamma + O\left(\frac{1}{m}\right) \\ E[A_{\alpha m}] &= \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \quad 0 \leq \alpha < 1; \end{aligned}$$

$$\begin{aligned}
V[A_n] &= \frac{2m^2}{n} (H_m^{(2)} - H_{m-n}^{(2)}) - \frac{m}{n} (H_m - H_{m-n}) - \frac{m^2}{n^2} (H_m - H_{m-n})^2 \\
V[A_m] &= 2H_m^{(2)}m - H_m - H_m^2 \\
&= \frac{\pi^2}{3}m - \ln^2 m - (1 + 2\gamma)\ln m - (2 + \gamma + \gamma^2) + O\left(\frac{1}{m}\right) \\
V[A_{\alpha m}] &= \frac{2}{1-\alpha} - \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) - \frac{1}{\alpha^2} \ln^2\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \\
&\qquad\qquad\qquad 0 \leq \alpha < 1.
\end{aligned}$$

As we can see, for most purposes, both models are crudely equivalent. They disagree for tables that are full (or nearly full), but even then the difference in the expected value is only one. The difference in the variance, however, is more important: in one case, the variance is essentially m , and in the other it is $(\pi^2/3)m$.

2. TABLE REORDERING SCHEMES

When an incoming element collides with another one that is occupying a given table slot, the standard open addressing method resolves the collision by using a first-come-first-served discipline: the element that arrived there first "wins," while the "loser" (the incoming element) continues probing. Amble and Knuth [1] used the fact that *any* of the colliding elements could validly be the winner (in the sense that the correctness of the search algorithm would be preserved) to speed up unsuccessful searches, by making the keys in any probe sequence be in increasing order (i.e., the element with the smallest key value is the winner).

Brent [2] introduced the idea of "looking ahead" before deciding which element should be the winner, and this was carried to its logical conclusion by Gonnet and Munro [8] and independently by Mallach [17]. By looking ahead, these methods are able to bring the expected search time down to a small constant (≈ 2.13 probes), even for a full table (at the cost of increasing the insertion time). The expected search time for tables under an optimal arrangement (both in the average and the min-max sense) has also been investigated [8, 9, 18].

Celis *et al.* [3, 4] introduced the idea of resolving collisions by comparing how far from their primary positions had each of the two colliding elements been moved and taking as the winner the one that had been moved the most. This blatant socialism was called "Robin Hood hashing." Even though the approach does not change the value of $E[A_n]$, relative to that of the standard insertion algorithm, it dramatically decreases the variance. For

a full table (by numerical extrapolation) $V[A_m]$ has been found to be approximately 1.883, a dramatic reduction from $O(m)$.

This fact can be utilized to replace the standard search algorithm by a *mean centered search*, where instead of probing positions $1, 2, \dots$ in the probe sequence of a key, we probe positions $k, k+1, k-1, k+2, k-2, \dots$, where k is the expected position where the key will be found in a table constructed with Robin Hood hashing. Celis *et al.* proved that this search runs in expected time $O(1)$ in a table loaded with $(1 - \epsilon)m$ elements, for any $\epsilon > 0$.

The main drawback of Robin Hood hashing is the need to determine, in every step of the insertion algorithm, how far from its primary position a given key is. This can be done by performing a division in a finite field (assuming we are using double hashing), which would introduce an extra $O(\log m)$ factor in the running time, for most machines. Another approach is to perform a mean centered search in the probe sequence. Although the latter takes time $O(1)$ on the average, it complicates the algorithm, and the constant factor it introduces may be unpleasant in practice.

3. THE "LAST-COME-FIRST-SERVED" HEURISTIC

In this paper, we propose the following much simpler heuristic: when an incoming element collides with another one already occupying a given table slot, the winner is the incoming element. Thus, the element stored in a table location is always the *last* one to have probed there.

This can be considered the dual of the standard (FCFS) algorithm, and looks, at first glance, like the same algorithm with the keys inserted in reverse order. We will see that this is not the case and that, analogously with what happens with Robin Hood hashing, $E[A_n]$ does not change, but $V[A_n]$ decreases dramatically.

Assuming random probing, all collision resolution methods that do not take into account the future probe sequences of the colliding records have the same expected successful search time [3],

$$E[A_n] = \frac{m}{n}(H_m - H_{m-n}).$$

This comes from the fact that, when an insertion is performed on a table with n elements, an average of $m/(m-n)$ slots will be probed until an empty one is found. This is the expected increase in the total accumulated search time, thus giving an average over all keys of

$$\frac{1}{n} \sum_{0 \leq k < n} \frac{m}{m-k} = \frac{m}{n} = \frac{m}{n}(H_m - H_{m-n}).$$

Under the standard FCFS method, the full $m/(m-n)$ cost must be borne by one element, the one being inserted. When n is close to m , this can be very high and leads to a large variance. Under the LCFS method, this cost is shared among many keys, thus giving some intuition as to why one might expect a low variance.

To analyze the performance of the LCFS method, we adopt the random probing model. (The analysis under the more realistic uniform probing model seems to be much harder and remains an open problem. In the light of the experimental data presented in Section 6, this does not seem to be a serious drawback.) We consider a table with $n-1$ keys and study the effect of the insertion of the n th key. For the purposes of the analysis, we number the keys according to their order of insertion into the table (K_1 is the first key inserted, K_2 the second one, etc.)

When key K_n first probes the table, it may collide with another key, say K_i . This happens with probability $1/m$ for each key in the table, for a total probability of $(n-1)/m$. Now key K_i in turn probes the table and may collide with yet another key, say K_j , again with probability $1/m$ for each key in the table. This "chain reaction" continues until an empty slot is found. On each probe, the probability of this happening is $(m+1-n)/m$.

To compute the increase in retrieval cost for each one of the keys, we introduce the probability generating function $R_n(x_1, \dots, x_n)$, where the coefficient of $x_1^{k_1} \cdots x_n^{k_n}$ is the joint probability that the increase in retrieval cost for K_1 is k_1 , for K_2 is k_2 , etc.

If we follow the chain reaction, we can write

$$R_n(x_1, \dots, x_n) = \sum x_{i_1} \frac{1}{m} x_{i_2} \frac{1}{m} \cdots \frac{1}{m} x_{i_s} \frac{m+1-n}{m},$$

where the sum is taken over all sequences $i_1, \dots, i_s \in \{1, \dots, n\}$ such that $s \geq 1$, $i_1 = n$, and $i_{j+1} \neq i_j$ ($0 \leq j < s$), since a key cannot collide with itself.

Let us define

$$S(x_1, \dots, x_n) = \sum x_{i_1} \cdots x_{i_s}$$

where the sum ranges over the same set as before. Then, clearly,

$$R_n(x_1, \dots, x_n) = (m+1-n) S\left(\frac{x_1}{m}, \dots, \frac{x_n}{m}\right).$$

The generating function $S(x_1, \dots, x_n)$ can easily be obtained using a formal language (or symbolic) approach [7, 11, 12], from the regular grammar

$$S \rightarrow T_n$$

$$T_i \rightarrow x_i(\varepsilon | T_1 | \cdots | T_{i-1} | T_{i+1} | \cdots | T_n) \quad (1 \leq i \leq n).$$

(Intuitively, T_i generates the set of all words that begin with x_i and such that no two adjacent letters are the same.)

From this, we can obtain a system of equations

$$S(x_1, \dots, x_n) = T_n(x_1, \dots, x_n)$$

$$T_i(x_1, \dots, x_n) = x_i \left(1 + \sum_{\substack{1 \leq j \leq n \\ j \neq i}} T_j(x_1, \dots, x_n) \right) \quad (1 \leq i \leq n),$$

where $T_i(x_1, \dots, x_n)$ represents the strings that begin with x_i and such that no two adjacent letters are the same. These strings correspond to terms in $S(x_1, \dots, x_n)$ for $i = n$.

The solution to these equations is

$$S(x_1, \dots, x_n) = \frac{x_n}{1 + x_n} \left(1 / \left(1 - \sum_{1 \leq j \leq n} \frac{x_j}{1 + x_j} \right) \right)$$

and therefore

$$R_n(x_1, \dots, x_n) = \frac{x_n}{m + x_n} \left((m + 1 - n) / \left(1 - \sum_{1 \leq j \leq n} \frac{x_j}{m + x_j} \right) \right).$$

Since the effect of each insertion is independent from that of previous insertions, the probability generating function $Q_n(x_1, \dots, x_n)$ for the retrieval cost for each key has a product form:

$$Q_n(x_1, \dots, x_n) = \prod_{1 \leq k \leq n} R_k(x_1, \dots, x_k)$$

$$= \prod_{1 \leq k \leq n} \left(\frac{x_k}{m + x_k} \left((m + 1 - k) / \left(1 - \sum_{1 \leq j \leq k} \frac{x_j}{m + x_j} \right) \right) \right)$$

The coefficient of $x_1^{k_1} \cdots x_n^{k_n}$ in $Q_n(x_1, \dots, x_n)$ represents the joint probability that in a LCFS table with n keys, the first key (in order of insertion) requires k_1 probes to be retrieved, the second one k_2 , and so on.

Finally, A_n , the cost of searching for a random element, has the probability generating function

$$P_n(z) = \frac{1}{n} (Q_n(z, 1, \dots, 1) + Q_n(1, z, 1, \dots, 1) + \cdots + Q_n(1, \dots, 1, z)).$$

With the aid of Maple [5], we can obtain for $P_n(z)$ the following surprisingly simple closed form:

$$P_n(z) = \frac{m+1}{n} \left(m^n / \left(\left(m+1 - z \frac{m+1}{m+z} \right)^n \right) \right) - \left(\frac{m+1}{n} - 1 \right).$$

The mean and the variance of A_n are obtained by differentiation. We summarize these results in the following theorem:

THEOREM 1. *The expected value of the number of probes to perform a successful search in a LCFS hash table, under the random probing model, is*

$$\mathbf{E}[A_n] = \frac{m}{n} (H_m - H_{m-n})$$

$$\mathbf{E}[A_m] = H_m = \ln m + \gamma + O\left(\frac{1}{m}\right)$$

$$\mathbf{E}[A_{\alpha m}] = \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \quad 0 \leq \alpha < 1,$$

and the variance is

$$\begin{aligned} \mathbf{V}[A_n] &= \frac{m(m-1)}{n(m+1)} (H_m - H_{m-n}) - \frac{(m-n+1)m^2}{(m+1)n^2} (H_m - H_{m-n})^2 \\ &\quad + \frac{m^2}{n(m+1)} (H_m^{(2)} - H_{m-n}^{(2)}) \end{aligned}$$

$$\begin{aligned} \mathbf{V}[A_m] &= \frac{m-1}{m+1} H_m - \frac{1}{m+1} H_m^2 + \frac{m}{m+1} H_m^{(2)} \\ &= \ln m + \gamma + \frac{\pi^2}{6} + O\left(\frac{\ln^2 m}{m}\right) \end{aligned}$$

$$\mathbf{V}[A_{\alpha m}] = \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) - \frac{1-\alpha}{\alpha^2} \ln^2\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right), \quad 0 \leq \alpha < 1.$$

Comparing with the corresponding expressions for the standard FCFS method with random probing, we observe that $\mathbf{E}[A_n]$ is the same (as expected), but $\mathbf{V}[A_n]$ is much smaller. For a full table, the behavior of $\mathbf{V}[A_m]$ is logarithmic instead of linear, and it differs from the expected value $\mathbf{E}[A_m]$ only in a constant term. Table 1 shows the asymptotic value of the variance for both methods (for a non-full table), for various values of α .

TABLE 1
 $V[A_{am}]$ for FCFS and LCFS with Random Probing

α	FCFS	LCFS
0.5	0.69	0.43
0.6	1.14	0.59
0.7	1.99	0.83
0.8	3.94	1.20
0.9	10.90	1.90
0.95	26.90	2.66
0.99	173.71	4.43
0.999	1945.27	6.87
0.9999	19905.94	9.20
0.99999	199855.94	11.51
0.999999	1999795.32	13.82
0.9999999	19999724.09	16.12

Before proceeding, we will study the asymptotic behavior of the distribution represented by $P_n(z)$. Define

$$\bar{P}_n(z) = \left(m^n / \left(\left(m + 1 - z \frac{m+1}{m+z} \right)^n \right) \right).$$

Then, we have that

$$P_n(z) = \frac{\bar{P}_n(z) - \bar{P}_n(0)}{1 - \bar{P}_n(0)};$$

i.e., the distribution whose probability generating function is $P_n(z)$ is a conditional distribution, obtained from $\bar{P}_n(z)$ by forbidding the zero state.

To simplify notation, let $\hat{z} = z((m+1)/(m+z))$. Then,

$$\begin{aligned} \bar{P}_n(z) &= \prod_{m-n < j \leq m} \left(\frac{1}{1 - (\hat{z} - 1)/j} \right) \\ &= \exp \left\{ \sum_{m-n < j \leq m} \ln \left(\frac{1}{1 - (\hat{z} - 1)/j} \right) \right\} \\ &= \exp \left\{ \sum_{m-n < j \leq m} \sum_{i \geq 1} \frac{(\hat{z} - 1)^i}{ij^i} \right\} \\ &= \exp \left\{ \sum_{i \geq 1} \frac{(\hat{z} - 1)^i}{i} (H_m^{(i)} - H_{m-n}^{(i)}) \right\}. \end{aligned}$$

Assume now that $n = \alpha m$ for some fixed parameter α ($0 \leq \alpha < 1$). Then,

$$H_m - H_{(1-\alpha)m} = \ln\left(\frac{1}{1-\alpha}\right) + O\left(\frac{1}{m}\right)$$

$$H_m(i) - H_{(1-\alpha)m}^{(i)} = O(m^{-(i-1)}) \quad (i > 1)$$

and, in a neighborhood of $z = 1$,

$$\hat{z} = z + O\left(\frac{1}{m}\right)$$

and

$$\bar{P}_{\alpha m}(z) = e^{(z-1)\ln(1/(1-\alpha))} + O\left(\frac{1}{m}\right) = \left(\frac{1}{1-\alpha}\right)^{z-1} + O\left(\frac{1}{m}\right).$$

This distribution whose probability generating function is $\bar{P}_{\alpha m}(z)$ is asymptotically Poisson with parameter $\lambda = \ln(1/(1-\alpha))$. Therefore, the asymptotic distribution of the random variable $A_{\alpha m}$ is positive Poisson with the same parameter, and its probability generating function is

$$P_{\alpha m}(z) = \frac{1}{\alpha} \left(\frac{1}{1-\alpha}\right)^{z-1} - \left(\frac{1}{\alpha} - 1\right) + O\left(\frac{1}{m}\right).$$

This asymptotic probability generating function is much more tractable than the exact one, and we will use it in the remainder of the paper. For most applications (e.g., for computing moments), this introduces an error that can be shown to be $O(1/m)$.

We can perform a similar asymptotic analysis for the joint distribution whose probability generating function is $Q_n(x_1, \dots, x_n)$. We first rewrite it as

$$Q_n(x_1, \dots, x_n) = \prod_{1 \leq k \leq n} \left(\hat{x}_k \left(1 / \left(1 - \frac{1}{m+1-k} \sum_{1 \leq j \leq k} (\hat{x}_j - 1) \right) \right) \right)$$

and then, working in a similar way as we did with $\bar{P}_n(z)$, we can prove that, in a neighborhood of $(x_1, \dots, x_n) = (1, \dots, 1)$, for $n = \alpha m$,

$$Q_{\alpha m}(x_1, \dots, x_{\alpha m}) = \prod_{1 \leq k \leq \alpha m} x_k e^{(x_k - 1)\ln((1 - (k-1)/m)/(1-\alpha))} + O\left(\frac{1}{m}\right).$$

Therefore, the number of probes to search for $K_1, \dots, K_{am}$ are asymptotically independent random variables. In particular, the number of probes to search for K_j is equal to one plus a random variable that follows a Poisson distribution with parameter

$$\lambda_j = \ln \left(\frac{1 - (j-1)/m}{1 - \alpha} \right).$$

4. A FASTER SEARCH ALGORITHM FOR LCFS HASHING

An important advantage of having a small variance is that the running time of the search algorithm is less subject to large random fluctuations. Furthermore, as mentioned in conjunction with Robin Hood hashing, a small variance implies we might profitably replace the standard search algorithm by a *centered search*.

It is well known that the optimal way to search a linear list of elements with unequal weights is in decreasing order of probability. In our case, we want to search in the linear list of locations $h_1(K), h_2(K), \dots$, where the probability of finding the element in location $h_k(K)$ follows (asymptotically) a positive Poisson distribution with parameter $\lambda = \ln(1/(1 - \alpha))$.

It is too complicated, and unnecessary, to try to follow the optimal order during the search. The following algorithm approximates the optimal order closely enough. The *mode* of a Poisson (λ) distribution is $\lfloor \lambda \rfloor$, and therefore the mode of the corresponding positive Poisson distribution is $\max(1, \lfloor \lambda \rfloor)$. If the mode is k , our algorithm probes location k first, and then $k+1$, $k-1$, $k+2$, $k-2$, etc.

Let M_n denote the number of probes for this mode-centered search. For $\lambda < 2$ (i.e., for $\alpha < 1 - e^{-2} \approx 0.86466$), the mode-centered algorithm is the same as the standard one (and also the same as the optimal algorithm), and

$$\mathbf{E}[M_{am}] = \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right) + O \left(\frac{1}{m} \right).$$

For $\lambda \geq 2$, we use the bound

$$\mathbf{E}[M_{am}] < 1 + 2\mathbf{E}[|A_n - \lfloor \lambda \rfloor|].$$

Let X be a random variable with distribution $\text{Poisson}(\lambda)$. Then,

$$\mathbf{E}[M_{am}] < 1 - 2 \frac{(1 - \alpha)}{\alpha} \lfloor \lambda \rfloor + \frac{2}{\alpha} \mathbf{E}[|X - \lfloor \lambda \rfloor|].$$

Let $\delta = \lambda - \lfloor \lambda \rfloor$. Now, to compute the expected absolute deviation of X

from $[\lambda]$, we expand and group similar terms, obtaining

$$\begin{aligned} E[|X - [\lambda]|] &= e^{-\lambda} \left\{ \sum_{0 \leq k \leq [\lambda]} (\lambda - \delta - k) \frac{\lambda^k}{k!} + \sum_{k \geq [\lambda]+1} (k - \lambda + \delta) \frac{\lambda^k}{k!} \right\} \\ &= 2e^{-\lambda} \frac{\lambda^{[\lambda]+1}}{[\lambda]!} + \delta \left(1 - 2e^{-\lambda} \sum_{0 \leq k \leq [\lambda]} \frac{\lambda^k}{k!} \right). \end{aligned}$$

We want to express this expected absolute deviation in terms of the standard deviation, $\sqrt{\lambda}$. Defining

$$r(\lambda) = 2e^{-\lambda} \frac{\lambda^{[\lambda]+1/2}}{[\lambda]!} + \frac{(\lambda - [\lambda])}{\sqrt{\lambda}} \left(1 - 2e^{-\lambda} \sum_{0 \leq k \leq [\lambda]} \frac{\lambda^k}{k!} \right),$$

we have that

$$E[M_{am}] < 1 - 2 \frac{(1 - \alpha)}{\alpha} [\lambda] + \frac{2}{\alpha} r(\lambda) \sqrt{\lambda}.$$

The function $r(\lambda)$ is discontinuous at all integer points, and for $\lambda \geq 2$ it reaches its maximum at $\lambda = 3^-$, where it takes the value $(10e^{-3} + 1)/\sqrt{3} \approx 0.864796$. As $\lambda \rightarrow \infty$ (i.e., as $\alpha \rightarrow 1$), the second term vanishes (it is $O(1/\sqrt{\lambda})$), while the first one approaches the limiting value $\sqrt{2/\pi} \approx 0.79788$ [6, 14]. (See Fig. 1.)

Replacing $r(\lambda)$ by its maximum value, we obtain the following result.

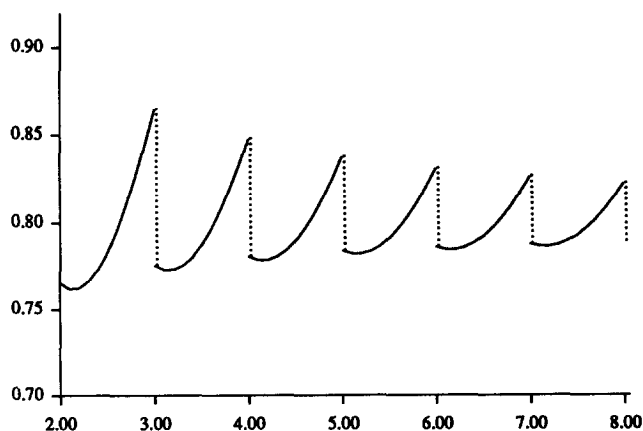


FIG. 1. The function $r(\lambda)$.

TABLE 2
Search Time for Different Search Algorithms

α	Standard	Mode-centered	Upper bound	Optimal
0.5	1.39	1.39	1.39	1.39
0.6	1.53	1.53	1.53	1.53
0.7	1.72	1.72	1.72	1.72
0.8	2.01	2.01	2.01	2.01
0.9	2.56	2.55	3.14	2.52
0.95	3.15	3.00	3.94	2.98
0.99	4.65	3.81	4.38	3.80
0.999	6.21	4.71	5.29	4.65
0.9999	9.21	5.34	5.79	5.31
0.99999	11.51	5.91	6.40	5.89
0.999999	13.82	6.46	6.98	6.41
0.9999999	16.11	6.92	7.37	6.89

THEOREM 2. *The expected number of probes for a successful mode-centered search in a LCFS hash table based on random probing is equal to*

$$\frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right) + O \left(\frac{1}{m} \right)$$

for $0 \leq \alpha < 0.86466$ and is bounded above by

$$1 - \frac{2(1 - \alpha)}{\alpha} \left\lfloor \ln \left(\frac{1}{1 - \alpha} \right) \right\rfloor + \frac{1.729592}{\alpha} \sqrt{\ln(1/(1 - \alpha))} + O \left(\frac{1}{m} \right)$$

for $0.86466 \leq \alpha < 1$.

This bound is not very tight, but it shows that, as the table becomes full, a mode-centered search reduces the running time of a search essentially to the square root of that of the standard algorithm. For actual computation, we will prefer the formula involving $r(\lambda)$. Table 2 shows the expected number of probes for the standard and the mode-centered search algorithms and the value of the bound. Also shown is the number of probes needed by the optimal algorithm (i.e., the algorithm that probes locations in decreasing order of probability).

5. THE EXPECTED WORST CASE OF LCFS HASHING

The random variable L_n gives us a measure of how bad a response time may get for some query. It is also useful for bounding unsuccessful searches. We wish to study its expected value $E[L_n]$.

Our asymptotic analysis indicates that the search time for each of the keys in the table is asymptotically equal to one plus a Poisson random variable. Furthermore, these random variables are asymptotically independent. Therefore, $E[L_{am}]$ is asymptotically equal to one plus the expected value of the maximum of a set of independent Poisson variables. The fact that each of these variables follows a Poisson distribution with a different parameter ($\lambda_1, \dots, \lambda_{am}$ in the notation of Section 3) complicates the analysis, but noting that $\lambda_1 > \lambda_2 > \dots$, we can see that $E[L_{am}]$ is bounded above by one plus the maximum of am independent Poisson variables, each with parameter $\lambda_1 = \ln(1/(1 - \alpha))$.

The maximum of a set of independent identically distributed Poisson variables has already been studied by Gonnet [9], who applied it to find $E[L_{am}]$ for hashing with separate chaining. Using Gonnet's formula (4.18), we immediately obtain the following theorem:

THEOREM 3. *The expected worst case for a successful search in a LCFS hash table with am keys based on random probing is bounded above by*

$$E[L_{am}] \leq 1 + \Gamma^{-1}(am) \left(1 + \frac{\ln \ln(1/(1 - \alpha))}{\ln \Gamma^{-1}(am)} + O\left(\frac{1}{\ln^2 \Gamma^{-1}(am)}\right) \right).$$

Although this is not a tight bound, the result is interesting because it proves that, for non-full tables, $E[L_{am}]$ grows sub-logarithmically in m . This is the first open addressing method for which such a bound has been proved, and it immediately implies that the same upper bound holds for a min-max hash table (a table under an optimal arrangement of the keys to minimize L_n) [8]:

COROLLARY 1. *For a min-max hash table with $n = am$ records, we have*

$$\left\lceil \frac{1}{\alpha} \ln\left(\frac{1}{1 - \alpha}\right) \right\rceil \leq E[L_{am}] \leq 1 + \Gamma^{-1}(am) \left(1 + \frac{\ln \ln(1/(1 - \alpha))}{\ln \Gamma^{-1}(am)} + O\left(\frac{1}{\ln^2 \Gamma^{-1}(am)}\right) \right).$$

Proof. The proof lower bound is from Gonnet [9]. By definition, $E[L_n]$ for a min-max hash table is less than or equal to $E[L_n]$ for any hashing scheme; in particular, it is bounded above by that of LCFS hashing. $\square$

6. SIMULATIONS

In addition to the analytic results obtained in the preceding sections, we ran a number of simulations of the LCFS algorithm. There are two main

TABLE 3
Simulation for a Table of Size $m = 1031$

α	$E[A_{am}]$	$V[A_{am}]$	$E[M_{am}]$	$E[L_{am}]$
0.5	1.3866 ± 0.0040	0.4256 ± 0.0061	1.3866 ± 0.0040	4.615 ± 0.090
0.6	1.5284 ± 0.0043	0.5922 ± 0.0072	1.5284 ± 0.0043	5.34 ± 0.10
0.7	1.7199 ± 0.0047	0.8309 ± 0.0087	1.7199 ± 0.0047	6.09 ± 0.11
0.8	2.0040 ± 0.0053	1.187 ± 0.011	2.004 ± 0.0053	7.06 ± 0.11
0.9	2.5552 ± 0.0063	1.888 ± 0.014	2.5186 ± 0.0064	8.53 ± 0.12
0.99	4.6358 ± 0.0092	4.480 ± 0.029	3.789 ± 0.010	12.91 ± 0.16
0.999	6.538 ± 0.012	7.118 ± 0.047	4.530 ± 0.013	16.27 ± 0.27

reasons for doing this. First, our analytical results assume the random probing model, while practical implementations often use the double hashing method, so there is a need for validation of the analytical predictions against experimental results. Second, there are still some open problems in the analysis of LCFS hashing, namely, the expected worst case and the running time for mode-centered searches in a full table. These simulations can be used to formulate conjectures about these performance measures.

We consider first the case of a non-full table. We simulated the LCFS algorithm on tables of size 1031 and 16411, for different values of α , repeating each experiment 200 times. The results are summarized in Tables 3 and 4.

In Figs. 2, 3, and 4 we have plotted this data together with the asymptotic performance predicted by our analysis. The graphs are presented as functions of $\lambda = \ln(1/(1 - \alpha))$ instead of α , so the behavior in a neighborhood of $\alpha = 1$ can be observed more easily. In all cases, there is good agreement between analysis and simulation.

TABLE 4
Simulation for a Table of Size $m = 16411$

α	$E[A_{am}]$	$V[A_{am}]$	$E[M_{am}]$	$E[L_{am}]$
0.5	1.38654 ± 0.00100	0.4250 ± 0.0015	1.38654 ± 0.00100	5.920 ± 0.085
0.6	1.5278 ± 0.0011	0.5943 ± 0.0018	1.5278 ± 0.0011	6.645 ± 0.096
0.7	1.7203 ± 0.0012	0.8329 ± 0.0022	1.7203 ± 0.0012	7.540 ± 0.094
0.8	2.0133 ± 0.0013	1.2029 ± 0.0027	2.0133 ± 0.0013	8.74 ± 0.11
0.9	2.5600 ± 0.0016	1.9056 ± 0.0037	2.5248 ± 0.0016	10.49 ± 0.12
0.99	4.6551 ± 0.0023	4.4465 ± 0.0072	3.7997 ± 0.0026	15.29 ± 0.14
0.999	6.9105 ± 0.0028	6.926 ± 0.011	4.6860 ± 0.0034	19.51 ± 0.16
0.9999	8.7059 ± 0.0033	9.108 ± 0.014	5.2133 ± 0.0038	22.56 ± 0.24

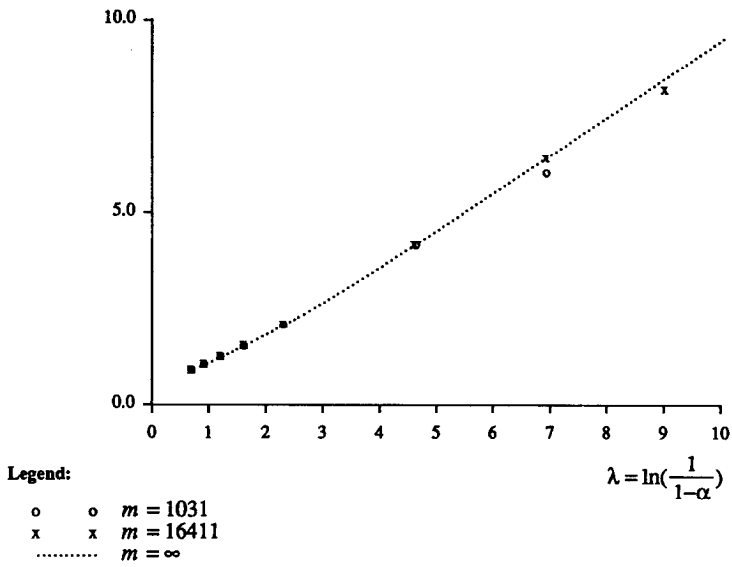


FIG. 2. Simulation results for $E[A_{am}]$.

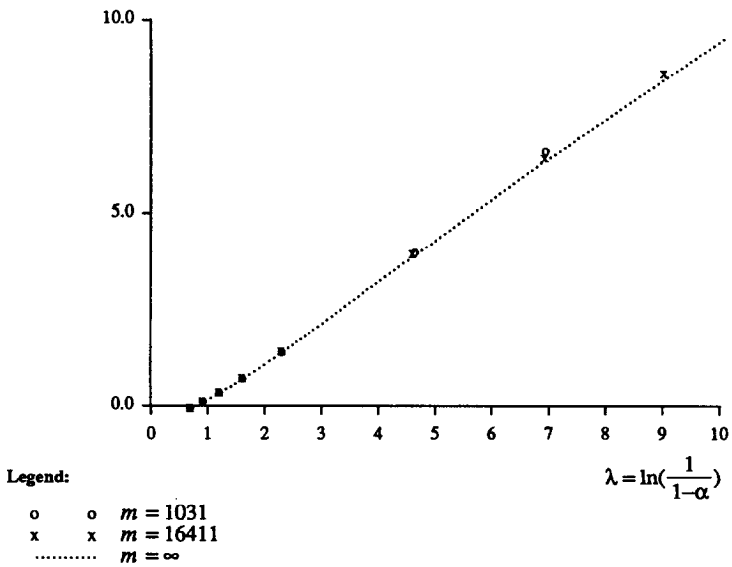


FIG. 3. Simulation results for $V[A_{am}]$.

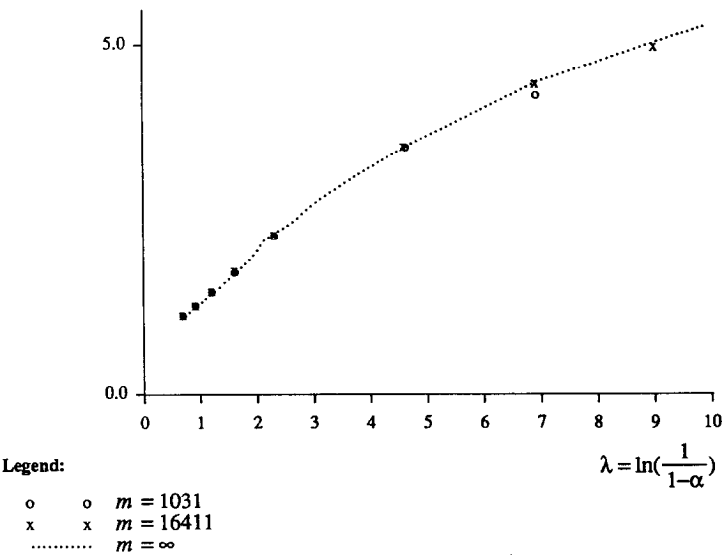


FIG. 4. Simulation results for $E[M_{am}]$.

TABLE 5
Simulation Results for a Full Table

m	$E[A_m]$	$V[A_m]$	$E[M_m]$	$E[L_m]$
3	1.473 ± 0.037	0.418 ± 0.033	1.378 ± 0.034	2.027 ± 0.068
5	1.872 ± 0.041	0.880 ± 0.056	1.717 ± 0.039	2.905 ± 0.086
11	2.660 ± 0.041	1.967 ± 0.097	2.341 ± 0.040	4.78 ± 0.13
17	3.122 ± 0.040	2.83 ± 0.11	2.712 ± 0.038	6.00 ± 0.16
37	3.864 ± 0.033	4.12 ± 0.12	3.243 ± 0.033	8.14 ± 0.20
67	4.635 ± 0.028	5.44 ± 0.11	3.674 ± 0.028	10.22 ± 0.21
131	5.261 ± 0.022	6.365 ± 0.092	3.997 ± 0.022	11.91 ± 0.22
257	6.079 ± 0.016	7.217 ± 0.068	4.322 ± 0.018	13.90 ± 0.23
521	6.812 ± 0.012	8.189 ± 0.055	4.603 ± 0.013	15.99 ± 0.25
1031	7.4430 ± 0.0092	9.204 ± 0.048	4.8210 ± 0.0100	17.77 ± 0.25
2053	8.2584 ± 0.0068	9.963 ± 0.034	5.0636 ± 0.0075	19.79 ± 0.24
4099	8.8532 ± 0.0049	10.370 ± 0.024	5.2449 ± 0.0055	21.50 ± 0.24
8209	9.6056 ± 0.0036	11.080 ± 0.018	5.4433 ± 0.0041	23.36 ± 0.22
16411	10.2391 ± 0.0026	11.808 ± 0.014	5.6075 ± 0.0030	25.13 ± 0.24
32771	10.9039 ± 0.0019	12.374 ± 0.010	5.7790 ± 0.0022	27.09 ± 0.24

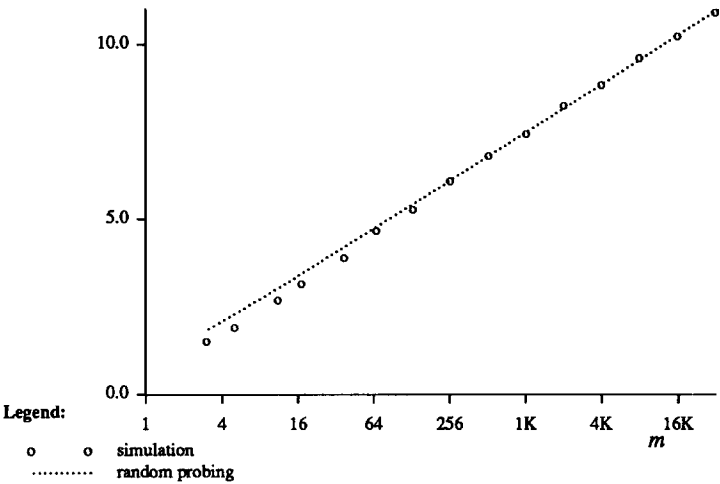


FIG. 5. $E[A_m]$.

We also simulated LCFS hashing on full tables, for m ranging from 3 to 32771. Each experiment was repeated 400 times, and the results are shown in Table 5.

The corresponding graphs are shown in Figures 5, 6, 7, and 8, using a logarithmic scale for m . For $E[A_m]$ and $V[A_m]$ we also plot the value predicted by the analysis.

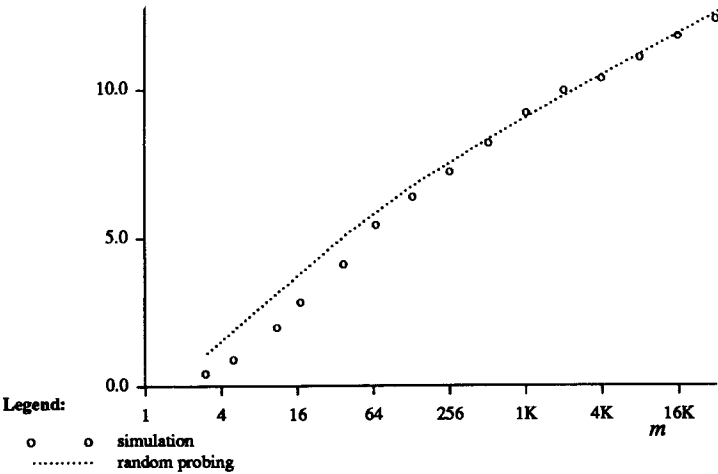
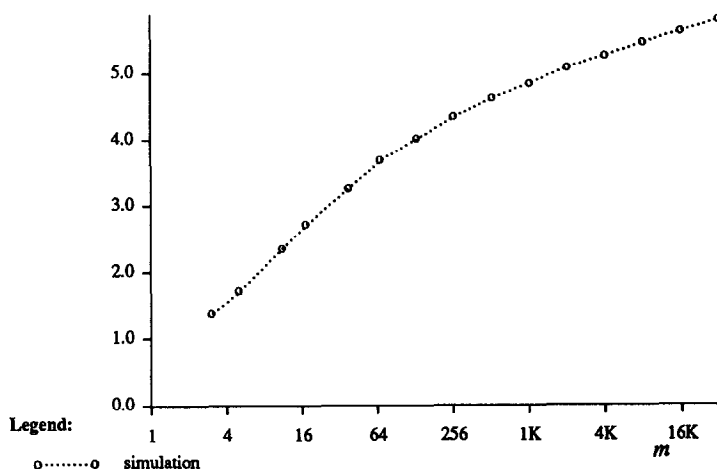
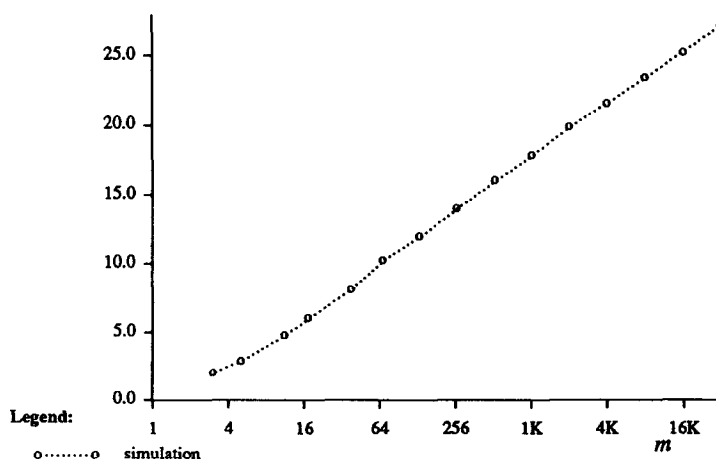


FIG. 6. $V[A_m]$.

FIG. 7. $E[M_m]$.

We can observe that in the case of a full table, random probing and double hashing are clearly different for small tables, but they become indistinguishable for large tables. Intuitively, double hashing is closer to uniform probing, since both sample the table without replacement. In FCFS hashing, the difference between sampling with or without replacement produces a variance that differs by a factor of more than three, but in LCFS hashing the difference can only be noticed in small tables.

FIG. 8. $E[L_m]$.

Figures 7 and 8 show the two performance measures for which no analytical results are available. From Fig. 7, it seems reasonable to conjecture that $E[M_m]$ is $O(\sqrt{\log m})$, while Fig. 8 suggests that $E[L_m]$ is $O(\log m)$.

7. CONCLUSIONS AND FURTHER WORK

We have introduced and analyzed a new hashing method, LCFS hashing, that compares favorably in some aspects with FCFS open addressing. This method is simple to implement and can be used to substantially decrease the variance of the search time, and, as a consequence, the search time itself.

We have obtained exact expressions for the main performance measures of the algorithm, as well as asymptotic expressions for the two important cases of an α -full table and a full table. Two problems still open in this area are the expected worst case and the running time for mode-centered searches in a full table.

This analysis has been performed under the random probing model. The performance of LCFS hashing under the uniform probing model is an open problem, but in light of our simulation results we do not expect it to be substantially different.

Several generalizations of the LCFS heuristic are possible, and we will consider them in a future paper. One of them uses a hybrid insertion method, where the new key has to probe k times unsuccessfully before entering "LCFS mode." The justification for doing this is that letting the new key stay in its first probe position is also a source of increased variance, so intuitively k should be chosen to be close to the expected successful search time. This should enhance the effectiveness of mode-centered search. Another generalization resolves collisions by flipping a biased coin. Both generalizations include LCFS and FCFS as particular cases.

REFERENCES

1. O. AMBLE AND D. E. KNUTH, Ordered hash tables, *Comput. J.* **17**, No. 2 (1974), 135–147.
2. R. P. BRENT, Reducing the retrieval time of scatter storage techniques, *Comm. ACM* **16**, No. 2 (1973), 105–109.
3. P. CELIS, "Robin Hood Hashing," Report CS-86-14, University of Waterloo, 1986.
4. P. CELIS, P.-A. LARSON, AND J. I. MUNRO, Robin Hood hashing, in "Proceedings, 26th IEEE Symposium on the Foundations of Computer Science, Oct. 1985," pp. 281–288.
5. B. W. CHAR, K. O. GEDDES, G. H. GONNET, AND S. M. WATT, "Maple User's Guide," Watcom, Waterloo, Ontario, 1985.
6. E. L. CROW, The mean deviation of the Poisson distribution, *Biometrika* **45** (1958), 444–453.

7. P. FLAJOLET, Elements of a general theory of combinatorial structures, in "Fundamentals of Computation Theory," Lecture Notes in Computer Science Vol. 199, pp. 112-127, Springer-Verlag, New York Berlin, 1985.
8. G. H. GONNET AND J. I. MUNRO, Efficient ordering of hash tables, *SIAM J. Comput.* **8**, No. 3 (1979), 463-478.
9. G. H. GONNET, Expected length of the longest probe sequence in hash code searching, *J. Assoc. Comput. Mach.* **28**, No. 2 (1981), 289-304.
10. G. H. GONNET, "Handbook of Algorithms and Data Structures," Addison-Wesley, London, 1984.
11. I. GOULDEN AND D. JACKSON, "Combinatorial Enumeration," Wiley, New York, 1983.
12. D. H. GREENE, "Labelled Formal Languages and Their Uses," Report STAN-CS-83-982, Stanford University, 1983.
13. L. J. GUIBAS AND E. SZEMEREDI, The analysis of double hashing, *J. Comput. System Sci.* **8**, No. 3 (1979), 463-478.
14. N. JOHNSON AND S. KOTZ, "Distributions in Statistics: Discrete Distributions," Houghton-Mifflin, Boston, 1969.
15. D. E. KNUTH, "The Art of Computer Programming, Vol. 3: Sorting and Searching," Addison-Wesley, Reading, MA, 1973.
16. G. E. LYON, Packed scatter tables, *Comm. ACM* **21**, No. 10 (1978), 857-865.
17. E. G. MALLACH, Scatter storage techniques: A unifying viewpoint and a method for reducing retrieval times, *Comput. J.* **20**, No. 2 (1977), 137-140.
18. R. L. RIVEST, Optimal arrangement of keys in a hash table, *J. Assoc. Comput. Mach.* **25**, No. 2 (1978), 200-209.